

一、复现别人的项目时：

1、首先下载 `docker desktop`，验证 `docker --version`。

2、`docker run hello-world`，在本地找 `hello-world` 镜像，没有则去云端仓库下载镜像。根据镜像开辟空间（在虚拟机中），启动一个容器，执行容器中预设好的脚本，打印欢迎，结束后容器进入停止状态。容器通过端口映射/网络通信与主机通信，使得主机能获取到容器的运行结果。

（`docker run` 命令包含如下几个命令：`docker pull` 拉取镜像，二进制、压缩分层的文件包；之后 `docker create` 创建容器，按照镜像中的配置配置好容器；最后 `docker start` 启动容器，按照 `dockerfile` 文件中的配置运行命令）

3、`docker run` 的常见参数：`-p` 端口映射，`8080: 80`，主机的 `8080` 端口访问容器的 `80` 端口。`-v`，容器中能看到挂载的主机中的文件。`--rm`，容器运行后自动删除。

4、`docker compose up -d`，`docker compose` 工具的 `up` 命令，一键启动多个容器。它会找到 `docker-compose.yml` 文件，运行文件中的多个镜像容器。

（相当于同时 `docker run` 多个容器，并且这几个容器间还可以相互通信，不需要配置网络）

1. 简单的 docker-compose.yml 示例

你可以想象这个文件就是一张“施工蓝图”：

```
YAML

version: '3.8'

services:
  # 服务一：我们的网页应用
  web-app:
    image: python:3.9-slim      # 镜像：去商店下载 Python 3.9
    ports:
      - "5000:5000"           # 端口：把主机的 5000 映射到容器的 5000
    volumes:
      - ./code:/app            # 挂载：把当前文件夹下的 code 映射到容器里的 app
    environment:
      - REDIS_HOST=redis-db     # 环境变量：告诉网页去找数据库
    depends_on:
      - redis-db               # 依赖：必须先启动数据库，再启动我

  # 服务二：存储数据的 Redis
  redis-db:
    image: redis:alpine        # 镜像：使用超轻量级的 Redis
    restart: always            # 重启策略：如果崩了，自动重启
```

有了 Docker Compose 之后：

- **网络自动化**：它会默认创建一个叫 `project_default` 的网络，把 `web-app` 和 `redis-db` 扔进去。
- **域名自动化**：在 `web-app` 容器里，你不需要知道 Redis 的 IP 地址，直接访问 `http://redis-db` 就能连上。这就是为什么我们在环境变量里写的是名字，而不是数字 IP。

虽然逻辑上等价，但在实际操作中，`docker-compose up` 解决了 `docker run` 的三大痛点：

1. “记忆力”痛点

如果你有 5 个容器（前端、后端、数据库、缓存、日志），每个容器都有 4-5 个参数。

- **使用 `docker run`**：你需要记住并输入 20 多个参数，错一个字母就跑不通。
- **使用 `docker-compose`**：参数全部写在文件里，文件就是你的“备忘录”。

2. “网络连通”痛点

在 `docker run` 中，如果你想让“后端容器”访问“数据库容器”，你需要手动创建网络，并在启动时指定网络。

而在 `docker-compose` 中，它会自动为文件里的所有服务创建一个**默认网络**。容器之间可以直接通过**服务名**（比如 `db` 或 `web`）相互访问，非常神奇。

3. “依赖顺序”痛点

`docker run` 无法感知先后顺序。如果你先启动了后端，后端去连数据库，发现数据库还没起来，后端可能就直接报错退出了。

在 Compose 中，你可以用 `depends_on` 明确告诉 Docker：“先启动数据库，等它稳了再启动我”。

二、打包自己的项目给别人用：

1、编写 dockerfile 和 docker-compose.yml：

Dockerfile

```
# 1. 基础环境
FROM python:3.9-slim

# 2. 设置工作目录
WORKDIR /app

# 3. 复制依赖清单并安装
# 注意：如果连接 MySQL，通常需要安装数据库驱动（如 pymysql 或 mysql-connector-python）
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 4. 复制项目所有代码
COPY . .

# 5. 启动程序
CMD ["python", "main.py"]
```

第二步：编写 docker-compose.yml（核心所在）

这是把代码和数据库“打包”在一起交给别人的关键文件。

YAML

```
version: '3.8'

services:
  # 服务一：你的代码
  app:
    build: . # 告诉 Docker 就在当前目录下找 Dockerfile 来构建
    ports:
      - "8080:8080"
    environment:
      - DB_HOST=db-server # 通过环境变量告诉代码如何找到数据库
      - DB_USER=root # 注意：这里的名字要和下面的服务名一致
      - DB_PASSWORD=secret
    depends_on:
      - db-server # 确保数据库先启动

  # 服务二：数据库
  db-server:
    image: mysql:8.0 # 直接用官方镜像，不用自己写 Dockerfile
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=secret
      - MYSQL_DATABASE=my_project
    volumes:
      - db_data:/var/lib/mysql # 确保数据库的数据重启后不会丢失

volumes:
  db_data: # 声明一个持久化数据卷
```

- **Dockerfile 是“内部视角”**：它描述的是**镜像内部**长什么样。比如：基于什么操作系统？装了哪些 Python 包？代码放在哪个目录？它是**构建 (Build)** 阶段的产物。
- **Docker Compose 是“外部视角”**：它描述的是**容器之间**如何相处。比如：对外暴露哪个端口？环境变量是什么？数据库密码是多少？它是**运行 (Run)** 阶段的配置。

理解为 docker-compose.yml 中的每一个容器，都有对应的环境配置文件 dockerfile。

2、Build 镜像

Build 出镜像，依次读取 dockerfile 的每一条指令，依次创建出新的“层”。相当于对一个虚拟环境进行环境配置。

3、push 到自己的 docker 仓库中，类似于自己的 github 仓库。

三、补充：

- 1、一个容器中只能运行一个镜像，但是一个项目启动时，可能通过 `docker-compose.yml` 同时运行着多个容器。
- 2、多个容器运行在一台虚拟机中。

Docker 利用了 Linux 内核的两大黑科技：

1. **Namespace (命名空间)**：给容器蒙上双眼。容器 A 以为自己是这台电脑上唯一的进程，看不到别的容器，也看不到主机的文件。
2. **Cgroups (控制组)**：给容器套上紧箍咒。规定容器 A 最多只能用 1GB 内存，不准抢占别人的资源。

4. 总结：一个项目多个容器的本质

当你通过 `docker-compose up` 启动一个包含 Web、MySQL、Redis 的项目时：

- 你的电脑里并没有多出 3 个操作系统。
- 你只是启动了 **3 个被高度隔离的特殊进程**。
- 这 3 个进程在主机的视角下，就像你同时打开了 Chrome、微信和网易云音乐一样，只是 Docker 帮它们各自建了一道“看不见的墙”。

3、docker 卷

docker卷是什么意思？

Docker 卷 (Docker Volume) 是 Docker 中用于**持久化数据**和**在容器之间共享数据**机制。

简单来说，它是 Docker 管理的一种存储在宿主机（你的电脑或服务器）上的文件或目录，但由 Docker 引擎专门管理，而不是由用户直接操作文件系统路径。

为什么需要 Docker 卷？

默认情况下，Docker 容器是**无状态**的。这意味着：

1. **数据易失性**：当容器被删除或重启时，容器内部产生的所有数据都会丢失。
2. **隔离性**：每个容器都有自己独立的文件系统，其他容器无法直接访问。

如果你运行一个数据库（如 MySQL、PostgreSQL）或者需要保存应用日志、用户上传的文件，你肯定不希望每次重启容器数据就没了。**Docker 卷就是为了解决这个问题而存在的。**

Docker 卷的核心特点

1. **持久化**：即使容器被删除，卷中的数据依然存在。
2. **独立于容器生命周期**：卷的存在不依赖于任何特定容器。你可以创建、挂载、删除卷，而不影响容器。
3. **易于备份和迁移**：因为卷由 Docker 管理，你可以轻松地备份、恢复或在不同容器间共享数据。
4. **性能更好**：相比“绑定挂载”（Bind Mount，即直接映射宿主机的某个具体路径），Docker 卷在 Linux 上通常具有更好的 I/O 性能，并且更安全（避免意外修改宿主机关键文件）。

4、docker 网络：

1. Bridge 网络（桥接模式）—— 默认且最常用

- **原理：**Docker 会在宿主机上创建一个虚拟网桥（通常叫 `docker0`）。所有连接到该网络的容器都会通过这个网桥进行通信。
- **特点：**
 - 每个容器会获得一个独立的内部 IP 地址（如 `172.17.0.2`）。
 - 容器之间可以通过 IP 或**容器名称**（在自定义网络中）互相访问。
 - 外部网络不能直接访问容器，除非通过**端口映射**（Port Mapping，如 `-p 8080:80`）。
- **适用场景：**大多数单机应用，如 Web 服务器连接数据库。

2. Host 网络（主机模式）

- **原理：**容器**不使用**独立的网络命名空间，而是直接使用宿主机的网络栈。
- **特点：**
 - 容器没有自己的 IP，直接使用宿主机的 IP。
 - 性能最高（没有 NAT 开销），但隔离性最差。
 - 端口冲突风险高（如果容器占用 80 端口，宿主机 80 端口也被占用）。
- **适用场景：**对网络性能要求极高，或者需要监听大量端口的应用。

四、案例：

一个项目运行时需要数据库，redis 等，docker-compose.yml 文件中定义了镜像名，环境变量，volumes（将数据同步保存到主机上，防止容器删除，导致数据丢失）。

1. PostgreSQL 数据库

```
! YAML
1 postgres:
2   container_name: supermew-postgres      # 容器名称
3   image: postgres:15                    # 使用 PostgreSQL 15 镜像
4   environment:
5     POSTGRES_DB: langchain_app           # 创建数据库名
6     POSTGRES_USER: postgres             # 用户名
7     POSTGRES_PASSWORD: postgres         # 密码
8   ports:
9     - "5432:5432"                       # 宿主机端口:容器端口
10  volumes:
11    - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/postgres:/var/lib/postgresql/data
12                                     # 数据持久化到本地 volumes 目录
13  healthcheck:
14    test: ["CMD-SHELL", "pg_isready -U postgres -d langchain_app"]
15                                     # 健康检查命令
16    interval: 10s                      # 每 10 秒检查一次
17    timeout: 5s                        # 超时时间
18    retries: 10                        # 重试 10 次
```

2. Redis 缓存

```
! YAML
1 redis:
2   container_name: supermew-redis
3   image: redis:7-alpine                 # 轻量版 Redis 7
4   command: ["redis-server", "--appendonly", "yes"]
5                                     # 开启 AOF 持久化
6   ports:
7     - "6379:6379"
8   volumes:
9     - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/redis:/data
10  healthcheck:
11    test: ["CMD", "redis-cli", "ping"]  # 用 ping 检查健康
```

五、菜鸟教程基本命令：

基本命令

实例

```
# 拉取镜像（如官方Nginx镜像）
docker pull nginx

# 运行容器（-d 后台运行，-p 映射端口）
docker run -d -p 80:80 nginx

# 查看运行中的容器
docker ps

# 构建镜像（基于当前目录的Dockerfile）
docker build -t my-app .

# 进入容器内部
docker exec -it <容器ID> /bin/bash
```

命令	功能	示例
docker run	启动一个新的容器并运行命令	docker run -d ubuntu
docker ps	列出当前正在运行的容器	docker ps
docker ps -a	列出所有容器（包括已停止的容器）	docker ps -a
docker build	使用 Dockerfile 构建镜像	docker build -t my-image .
docker images	列出本地存储的所有镜像	docker images
docker pull	从 Docker 仓库拉取镜像	docker pull ubuntu
docker push	将镜像推送到 Docker 仓库	docker push my-image
docker exec	在运行的容器中执行命令	docker exec -it container_name bash
docker stop	停止一个或多个容器	docker stop container_name
docker start	启动已停止的容器	docker start container_name
docker restart	重启一个容器	docker restart container_name
docker rm	删除一个或多个容器	docker rm container_name
docker rmi	删除一个或多个镜像	docker rmi my-image
docker logs	查看容器的日志	docker logs container_name
docker inspect	获取容器或镜像的详细信息	docker inspect container_name
docker exec -it	进入容器的交互式终端	docker exec -it container_name /bin/bash
docker network ls	列出所有 Docker 网络	docker network ls
docker volume ls	列出所有 Docker 卷	docker volume ls
docker-compose up	启动多容器应用（从 docker-compose.yml 文件）	docker-compose up
docker-compose down	停止并删除由 docker-compose 启动的容器、网络等	docker-compose down
docker info	显示 Docker 系统的详细信息	docker info
docker version	显示 Docker 客户端和守护进程的版本信息	docker version
docker stats	显示容器的实时资源使用情况	docker stats